

FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA

Gabriel Lacerda Covello Arimatéa 556391

Geovana Carvalho Pederneschi 559092

Gustavo Andrade de Sousa 559069

Lucas Santos Rodrigues 556891

Mayene Gabrielle Aragão Padilha Doria 558858

Thais Helena Ferreira Vieira 552387

RELATÓRIO
Mentoria 4 – Documentação consolidada

ENGENHARIA DA COMPUTAÇÃO - 3ECR
SPI/Metaindústria

SUMÁRIO

1. Introdução 4
1.1 Contextualização da fase 4
1.2 Histórico das mentorias 4
1.3 Requisitos mínimos 4

2. Machine Learning 6
2.1 Desenvolvimento 6
2.1.1 Revisão teórica 6
2.1.2 Dados 6
2.1.3 Metodologia 7
2.1.4 Ferramentas 8
2.1.5 Pipeline de implementação 9
2.2 Resultados 10
2.2.1 Métricas de avaliação 10
2.2.2 Desempenho do(s) modelo(s) 10
2.2.3 Análise de erros 10
2.3 Evidências 11
2.3.1 Visualizações de suporte 11
2.3.2 Exemplos práticos 11
2.3.3 Logs e reprodutibilidade 12

3. Frontend 14
3.1 Desenvolvimento 14
3.1.1 Revisão teórica 14
3.1.2 Requisitos e escopo da interface 14
3.1.3 Arquitetura e componentes 14
3.1.4 Decisões de UX e novas telas 14
3.1.5 Integração com backend 16
3.1.6 Ferramentas 16
3.1.7 Pipeline de implementação 16
3.2 Resultados 16
3.2.1 Critérios de avaliação 16
3.2.2 Funcionalidades entregues 17

3.2.3 Análise de problemas . . . 17
3.3 Evidências . . . 17
3.3.1 Prints e protótipos de telas . . . 17
3.3.2 Exemplos práticos . . . 17
3.3.3 Logs e reprodutibilidade . . . 17

4. Backend . . . 18
4.1 Desenvolvimento . . . 18
4.1.1 Revisão teórica . . . 18
4.1.2 Requisitos e escopo dos serviços . . . 18
4.1.3 Arquitetura . . . 18
4.1.4 Integração com o modelo de ML . . . 18
4.1.5 Integração com hardware (ESP32) . . . 19
4.1.6 Ferramentas . . . 19
4.1.7 Pipeline de implementação . . . 20
4.2 Resultados . . . 20
4.2.1 Métricas de avaliação . . . 20
4.2.2 Desempenho dos serviços . . . 20
4.2.3 Análise de erros . . . 20
4.3 Evidências . . . 20
4.3.1 Visualizações de suporte . . . 20
4.3.2 Exemplos práticos . . . 21
4.3.3 Logs e reprodutibilidade . . . 21

5. Conclusão . . . 22
5.1 Considerações finais . . . 22

1. Introdução

1.1 Contextualização da fase

O projeto Codexis nasce da constatação de que a fiscalização manual de segurança do trabalho não acompanha a realidade das linhas de produção industriais. Em 2023, o Brasil registrou mais de 580 mil acidentes de trabalho, dos quais aproximadamente 30% envolveram falha no uso correto de Equipamentos de Proteção Individual (EPIs), gerando um custo direto e indireto estimado em R$ 70 bilhões por ano com Segurança e Saúde no Trabalho (SST), segundo dados do Ministério do Trabalho e Emprego (MTE/Smartlab) e da Organização Internacional do Trabalho (OIT).

Essa ineficiência decorre de quatro fatores estruturais: inspeções presenciais pontuais que não cobrem turnos completos; câmeras de CFTV que apenas gravam, sem qualquer análise automatizada; registros e auditorias dependentes de planilhas e papel; e identificação tardia de não conformidades, quando o acidente já ocorreu.

Este relatório (Mentoria 4) consolida a documentação produzida nas três mentorias anteriores — reconhecimento de EPIs e dashboard (Mentoria 1), pose estimation e análise comportamental (Mentoria 2) e decisões de UX (Mentoria 3) — em um único documento organizado por frente técnica (Machine Learning, Frontend e Backend), e acrescenta as quatro frentes de evolução definidas para esta etapa: ampliação do dataset de detecção de EPI, melhoria da latência do streaming da câmera, telas de auditoria e inventário de incidentes e notificação visual e sonora com componentes físicos (ESP32). O sistema é referido nas mentorias como Codexis (nome do grupo e do projeto) e SafeVision (nome do sistema no relatório técnico da Fase 2); ambos designam a mesma solução.

1.2 Histórico das mentorias

Mentoria	Tema	Principais entregas
1ª mentoria (04/05/2026)	Detecção de EPI (Fase 1)	Detecção de pessoa, capacete, máscara e colete com YOLOv8; API Flask; dashboard inicial com central de logs, gráficos e matriz de confusão.
2ª mentoria (22/06/2026)	Pose Estimation e Análise Comportamental (Fase 2)	YOLOv8-Pose (17 keypoints); score REBA; detecção de queda; verificação de zona crítica (Shapely); orquestrador unificado com debounce e suporte a dual-câmera.
3ª mentoria (19/08/2026)	Documentação de UX (Fase 3)	Decisões de canal, escala de severidade dos alertas, linguagem da interface e tecnologias de frontend (React, Tailwind CSS, Zustand, Axios).
4ª mentoria (23/09/2026)	Consolidação e evolução	Relatório consolidado; ampliação do dataset de EPI; redução de latência do streaming; telas de auditoria e inventário de incidentes; sinalização física com ESP32.

1.3 Requisitos mínimos

Para o Challenge, foram definidos os seguintes requisitos mínimos do MVP:

- Detecção de humanos e EPIs (capacete, óculos, colete, máscara, bota e protetor auricular);
- Pipeline de coleta, anotação (labeling) e treinamento de modelo YOLOv8;
- API em Flask para inferência do modelo, com geração de logs em PDF/CSV;
- Front-end web com transmissão de câmera ao vivo, gerenciamento e dashboard de KPIs;
- Análise ergonômica (REBA), detecção de queda e verificação de invasão de zona crítica;
- Demonstração funcional com câmera USB ou vídeo simulado de chão de fábrica.

MENTORIA 4 · NOVO NO ESCOPO
Ampliação do dataset de EPI; otimização da latência do streaming; telas de auditoria e inventário de incidentes; sinalização visual e sonora física com ESP32.

2. Machine Learning

2.1 Desenvolvimento

2.1.1 Revisão teórica

A frente de Machine Learning parte da experiência já validada pelo grupo no Challenge Aché 2025, reaproveitando uma stack de visão computacional testada anteriormente: detecção de objetos com YOLOv8, back-end em Flask e geração automatizada de logs. Essa continuidade técnica reduz o risco de implementação e permite concentrar o esforço no novo domínio de segurança ocupacional industrial.

Na Fase 2, a análise postural foi fundamentada no método REBA (Rapid Entire Body Assessment), escolhido em detrimento do RULA por avaliar o corpo inteiro, incluindo pernas, o que é mais adequado às posturas dinâmicas e em pé típicas do chão de fábrica (o RULA é focado em membros superiores e cenários sentados). As referências normativas são: NR-17 (Ergonomia), NR-35 (Trabalho em Altura), ISO 11226 (limites para posturas estáticas de trabalho) e o próprio REBA (Tabelas A e C oficiais).

2.1.2 Dados

O dataset de EPI é composto por imagens de trabalhadores em ambiente simulado de chão de fábrica, anotadas com bounding boxes para a classe Person (pessoa) e para os seis EPIs finais do projeto: capacete, óculos, colete, máscara, bota e protetor auricular, cada um com as variantes de presença e ausência (por exemplo, Hardhat / NO-Hardhat). A anotação é realizada na ferramenta Roboflow, também utilizada para versionamento e exportação do dataset no formato compatível com o treinamento YOLOv8.

O extrator de pose (YOLOv8-Pose) utiliza pesos pré-treinados da Ultralytics sobre o dataset público COCO Keypoints, sem fine-tuning adicional. Conforme a Política de Datasets do Guia de Regras, datasets públicos podem ser base do trabalho, mas o protótipo final deve ser validado também em dados produzidos pela própria equipe, com termo de consentimento dos voluntários.

> **MENTORIA 4 · AMPLIAÇÃO DO DATASET DE EPI**
> Nesta etapa, o dataset de detecção de EPI passa a ser ampliado com imagens reais de chão de fábrica e com maior variedade de condições de captura, priorizando os pontos de erro identificados na análise de resultados (confusão entre colete e óculos).

- **Diversidade de cenas:** poses, distâncias e ângulos de câmera variados; diferentes níveis de iluminação; oclusões parciais e mais de uma pessoa por quadro;
- **Classes finais de EPI:** capacete, óculos, colete, máscara, bota e protetor auricular;
- **Balanceamento:** equilíbrio entre exemplos de EPI presente e ausente (NO-Hardhat, NO-Mask, NO-Safety Vest) para cada classe;
- **Divisão treino/validação/teste:** separação feita por cena ou vídeo, e não por quadro, para evitar vazamento entre os conjuntos e permitir métricas confiáveis;
- **Data augmentation** no Roboflow (variação de brilho, desfoque e rotações leves) aplicado somente ao conjunto de treino;

- **Versionamento e privacidade:** cada versão do dataset registrada no Roboflow e referenciada no GitHub; termo de consentimento para pessoas gravadas e minimização dos dados armazenados.

O dataset final é composto por 1800 imagens, divididas em treino (60%), validação (25%) e teste (15%):

Conjunto	Nº de imagens	Nº médio de instâncias por classe
Treino (60%)	1080	1150
Validação (25%)	450	480
Teste (15%)	270	290

2.1.3 Metodologia

O algoritmo escolhido para EPI foi o YOLOv8 (Ultralytics), por seu equilíbrio entre velocidade de inferência e precisão, adequado a aplicações de detecção em tempo real — critério decisivo para um sistema de monitoramento contínuo. O modelo é fine-tuned a partir de pesos pré-treinados, com a arquitetura de detecção original mantida e a camada de classificação ajustada às classes do projeto.

A estratégia de treinamento segue as etapas: (1) setup e inferência inicial do modelo base; (2) labeling e definição de bounding boxes; (3) treino e testes do modelo de detecção de EPI; (4) modelo complementar de risco ergonômico; (5) modelo/módulo de zona de perigo; (6) ciclo de testes e retreino; (7) validação; e (8) otimização de latência para uso em tempo real.

Pose e ergonomia. Foi adotado o YOLOv8-Pose (variante nano, yolov8n-pose.pt), com 17 keypoints no padrão COCO, atendendo ao requisito mínimo do Descritivo Técnico. A escolha reaproveita o framework Ultralytics da Fase 1, prioriza inferência em tempo real (três pipelines simultâneos sobre o mesmo frame) e reduz o risco de integração pela maturidade do modelo. A partir dos keypoints são calculados quatro valores por pessoa:

Grandeza	Keypoints (índice COCO)	Interpretação
Ângulo de tronco	Ombro (5/6) → Quadril (11/12) → Joelho (13/14)	Flexão/extensão da coluna
Ângulo de pescoço	Nariz (0) → Ombro (5/6) → Quadril (11/12)	Flexão cervical
Ângulo de pernas	Quadril (11/12) → Joelho (13/14) → Tornozelo (15/16)	Flexão de joelho (agachamento)
Assimetria de ombros	Ombro esq. (5) vs. ombro dir. (6): diferença de altura em px	Torção/carga lateral de tronco

Para cada cálculo de ângulo, o sistema usa o lado do corpo (esquerdo ou direito) com maior confiança média de detecção e, quando o quadril não está visível, recorre a um cálculo geométrico alternativo (ponto virtual abaixo da linha dos ombros), evitando descartar leituras em cenas parcialmente ocluídas. Cada ângulo é convertido em subscore segundo faixas adaptadas da escala REBA:

Segmento	Faixa do ângulo	Score	Classificação
Tronco	> 170°	1	Neutro

Segmento	Faixa do ângulo	Score	Classificação
Tronco	150° – 170°	2	Leve
Tronco	120° – 149°	3	Moderado
Tronco	90° – 119°	4	Acentuado
Tronco	< 90°	5	Severo (risco imediato)
Pescoço	> 160°	1	Neutro
Pescoço	140° – 160°	2	Leve
Pescoço	< 140°	3	Excessivo
Pernas	> 160°	1	Estendido
Pernas	120° – 160°	2	Leve flexão
Pernas	90° – 119°	3	Agachamento parcial
Pernas	< 90°	4	Agachamento acentuado

Os subscores de tronco, pescoço e pernas são combinados pela Tabela A do REBA (score de grupo A, de 1 a 9), que é cruzado com o score de assimetria de ombros na Tabela C, gerando o score final (1 a 15):

Score REBA	Nível	Ação recomendada
1 – 3	BAIXO	Postura aceitável; nenhuma ação necessária
4 – 7	MÉDIO	Investigar; correção recomendada em médio prazo
8 – 15	ALTO	Ação necessária em breve / imediatamente

Detecção de queda. Heurística de risco imediato com dois critérios independentes: (1) nariz abaixo da linha dos ombros; e (2) proporção largura/altura do bounding box maior que 1,5 (corpo na horizontal), validada apenas quando ombro, quadril e ao menos um ponto de perna estão visíveis com confiança ≥ 0,5, para não confundir enquadramento cortado com queda real. Por ser risco imediato, o sistema prioriza recall sobre precision nesse cenário.

Zona crítica. O módulo ZoneChecker usa geometria computacional (biblioteca Shapely): cada câmera pode ter uma zona poligonal em coordenadas de pixel, e a presença de qualquer um dos treze keypoints monitorados (tornozelos, joelhos, quadris, ombros, nariz, punhos e cotovelos) dentro do polígono é reportada como invasão. Cada zona pode carregar uma lista de EPIs obrigatórios (epis_obrigatorios), permitindo que a regra de conformidade varie por área (por exemplo, capacete e colete apenas na “área da prensa”).

2.1.4 Ferramentas

- **IA / Inferência:** Python · OpenCV (captura e pré-processamento de frames) · Shapely (geometria de zona) · PyAV (captura RTSP);
- **Orquestração:** orquestrador central em Python · biblioteca websockets (streaming em tempo real, porta 8765);
- **Dataset / Labels:** YOLOv11 · Roboflow · Ultralytics — rotulagem e exportação do dataset customizado de EPI (o extrator de pose utiliza o YOLOv8-Pose da Ultralytics).

2.1.5 Pipeline de implementação

O fluxo de inferência segue quatro camadas, conforme o fluxograma do processo (seção 4.3.1): a camada de Sistema de Captura conecta-se à câmera e captura o frame; a camada de Machine Learning executa o modelo treinado e retorna a classificação (EPI detectado ou não); a camada de Interface Visual atualiza a bounding box com o rótulo OK ou NOK e, em caso de não conformidade, dispara alerta visual e sonoro; por fim, a camada de Sistema de Log registra os dados, salva o frame localmente e armazena as informações no banco de dados, retroalimentando os gráficos do dashboard.

Na Fase 2, os três pipelines (EPIDetector, PoseAnalyzer e ZoneChecker) foram unificados sob um orquestrador central (orquestrador/main.py), que captura o frame uma única vez e o distribui aos três detectores; combina os resultados em um veredicto único por frame (MONITORANDO, ALERTA_EPI, ALERTA_ERGONOMIA, ALERTA_ZONA ou ALERTA_MULTIPLO); aplica debounce independente por tipo de risco; e transmite o resultado ao frontend via WebSocket, registrando incidentes confirmados pela API REST.

Tipo de risco	Frames p/ confirmar	Cooldown (frames)
EPI (Fase 1)	10	60
Ergonomia (REBA ≥ 4)	8	60
Zona crítica	3	30
Queda (risco imediato)	6	120

O sistema suporta captura dual-câmera: uma câmera frontal (Fase 1, EPI) e uma lateral (Fase 2, ergonomia/zona), com dois modos de fusão configuráveis — “independente” (cada câmera observa um posto distinto) e “mesma_pessoa” (as leituras REBA das duas visões são fundidas para maior robustez).

MENTORIA 4 · LATÊNCIA DO STREAMING DA CÂMERA
A melhoria da latência da transmissão de câmera é o foco de desempenho desta etapa. A meta de referência do Descritivo Técnico é latência ponta a ponta (captura → classificação) de até 200 ms. As ações previstas são:

- **Instrumentação por etapa:** medir separadamente captura, inferência de EPI, pose, zona, codificação do frame, transporte e renderização, para localizar o gargalo antes de otimizar;
- **Captura em thread dedicada** com buffer mínimo, descartando frames antigos para que a inferência sempre trabalhe no frame mais recente;
- **Redução da resolução** enviada à inferência e compressão JPEG com qualidade ajustável no streaming;
- **Inferência espaçada:** executar os modelos a cada N frames e reaproveitar a última detecção nos intermediários, mantendo o vídeo fluido;
- **Otimização do modelo:** exportação (por exemplo, ONNX) e uso de precisão reduzida/GPU quando disponíveis no hardware-alvo;
- **Transporte:** manter o WebSocket já adotado no orquestrador, evitando polling HTTP, e enviar apenas metadados leves das detecções quando o vídeo já está no navegador.

2.2 Resultados

2.2.1 Métricas de avaliação

Na Fase 1, o desempenho é acompanhado por matriz de confusão por classe de EPI, disponível no dashboard. O Guia de Regras exige, como escopo mínimo da Fase 2, o reporte de PCK (Percent of Correct Keypoints), acurácia de classificação postural e recall em postura de risco. As metas de referência e o campo para os valores medidos são:

Métrica	Meta sugerida (Descritivo Técnico)	Valor obtido
PCK (t = 0,1)	≥ 0,80	[A PREENCHER]
Acurácia de classificação postural	≥ 0,85	[A PREENCHER]
Recall em postura de risco imediato	≥ 0,90	[A PREENCHER]
Falsos alarmes / hora	Tendendo a zero	[A PREENCHER]
Latência E2E (captura → classificação)	≤ 200 ms	[A PREENCHER]

Esses valores precisam ser medidos empiricamente sobre um conjunto de vídeos de validação com gabarito (mínimo de três cenários distintos, conforme o item 5.5 do Descritivo Técnico). A branch analisada implementa a lógica de classificação e os mecanismos de debounce, mas ainda não contém um script de avaliação quantitativa.

2.2.2 Desempenho do(s) modelo(s)

Nos testes exploratórios de inferência (demonstração em tempo real via webcam), o modelo apresentou os seguintes níveis de confiança: detecção de pessoa (0,72–0,85), capacete presente (0,86) e ausente (0,89), máscara presente (0,85) e ausente (0,92), e colete ausente (0,54–0,82). Na matriz de confusão do dashboard, as classes capacete e óculos atingiram 95% de acerto, enquanto a classe colete atingiu 82%. A análise da Fase 1 é qualitativa e serve como baseline para as rodadas de retreino.

2.2.3 Análise de erros

A matriz de confusão indica maior confusão entre as classes colete e óculos (13% de erro cruzado), sugerindo sobreposição visual entre esses elementos em determinados ângulos de câmera ou distância. Esse ponto é a prioridade das rodadas de retreino, com a ampliação do dataset descrita na seção 2.1.2.

Casos de borda já tratados na Fase 2:

Caso de borda	Estratégia de mitigação
Quadril não visível (oclusão parcial)	Fallback geométrico: ponto virtual abaixo da linha dos ombros para estimar tronco/pescoço.
Poucos keypoints visíveis (< 3)	REBA retorna score mínimo (1, BAIXO) com motivo “keypoints_insuficientes” no log, sem gerar falso alerta.
Enquadramento cortado (webcam)	Queda exige confiança ≥ 0,5 em ombro, quadril e perna antes de validar o critério de bounding box horizontal.
Leitura ruidosa em um único frame	Debounce por N frames consecutivos (8 ergonomia, 6 queda, 3 zona).
Assimetria entre lados do corpo	Escolha do lado (E/D) com maior confiança média de detecção.

Limitações conhecidas: múltiplas pessoas sobrepostas na mesma região da imagem, iluminação muito baixa (reduz a confiança de todos os keypoints) e ausência de dataset próprio anotado para validação quantitativa das métricas de pose.

2.3 Evidências

2.3.1 Visualizações de suporte

O dashboard reúne quatro visualizações principais de suporte à análise do modelo: um gráfico de análise composta (série temporal de detecções ao longo do dia), um gráfico de detecções em formato de rosca (prevalência por classe), um radar de eficiência operacional (precisão da IA, estabilidade, conexão com o hardware, tempo de resposta e segurança/EPI) e a matriz de confusão por classe.



Figura 1 - Dashboard com central de logs, gráficos de detecção e matriz de confusão.

2.3.2 Exemplos práticos

As imagens a seguir mostram exemplos reais de inferência do modelo em vídeo capturado por webcam, com bounding boxes e scores de confiança para cada classe detectada.



Figura 2 - Detecção de capacete e máscara presentes, colete ausente (NO-Safety Vest).



Figura 3 - Detecção de ausência de capacete, máscara e colete (cenário de não conformidade).

2.3.3 Logs e reprodutibilidade

Cada evento de detecção é registrado com timestamp na central de logs do dashboard (ex.: “Operário detectado sem capacete na Zona B”), permitindo rastreabilidade e geração de relatório via botão dedicado. O código do modelo, o dataset e as configurações experimentais são versionados no GitHub do grupo, e o planejamento das rodadas de treino é documentado no Miro (roadmap do projeto).

IA generativa. Não foi identificado, nos módulos da Fase 2, uso de IA generativa para aumento sintético de dados. Caso o grupo utilize LLMs como apoio ao desenvolvimento, o Guia de Regras exige documentar em quais arquivos/funções e com que finalidade.

Privacidade e LGPD. O processamento de pose e zona opera sobre keypoints (coordenadas numéricas), sem reconhecimento facial nem associação a identificadores pessoais. Como o frame de origem contém imagens de pessoas reais, aplicam-se: consentimento explícito de quem for gravado; armazenamento mínimo de frames de

evidência, com controle de acesso; e definição de prazo e critério de exclusão dos dados ao final do projeto.

3. Frontend

3.1 Desenvolvimento

3.1.1 Revisão teórica

A frente de Frontend reaproveita padrões de interface já validados no Challenge Aché 2025, como telas de transmissão ao vivo e telas de gerenciamento de API, adaptando-os ao contexto de segurança ocupacional industrial. As decisões de UX seguem princípios de usabilidade, redução de carga cognitiva, hierarquia visual e rapidez na tomada de decisão em ambientes industriais.

3.1.2 Requisitos e escopo da interface

O escopo de interface contempla: definição de identidade visual no Figma; tela web principal (index); dashboard com aba de logs; atualização contínua de gráficos conforme novos dados de inferência; e tela de câmera dedicada à zona de perigo. Os objetivos de experiência do usuário são:

- Facilitar o monitoramento das câmeras em tempo real;
- Permitir rápida identificação de falhas operacionais;
- Destacar informações relevantes para segurança;
- Reduzir o tempo necessário para interpretar os dados do sistema;
- Centralizar todas as informações em um único ambiente;
- Melhorar a eficiência operacional do usuário.

> **MENTORIA 4 · TELAS DE AUDITORIA E INVENTÁRIO DE INCIDENTES**
> O escopo de interface passa a incluir telas para consulta, auditoria e tratamento dos incidentes registrados pelo sistema (seção 3.1.4).

3.1.3 Arquitetura e componentes

A interface é organizada em torno de dois blocos: (1) a tela de câmera ao vivo, com sobreposição das bounding boxes e um painel lateral de “Detecção de EPIs”, no qual o usuário seleciona via checkboxes quais equipamentos monitorar (colete, óculos, capacete, máscara), além de um painel de “Atenção” com os percentuais de confiança; e (2) o dashboard de gestão, estruturado em cards — central de logs, análise composta, gráfico de detecções, eficiência operacional e matriz de confusão — com botão de geração de relatório.

3.1.4 Decisões de UX e novas telas

Canais de comunicação. O canal visual é o principal meio de comunicação do sistema, pois permite ao operador interpretar informações rapidamente, sem leitura extensa (cards informativos, indicadores numéricos, gráficos, estados visuais, cores de status e painéis de monitoramento). Os demais canais são:

Canal	Elementos	Justificativa
Estados operacionais	ONLINE, OFFLINE, SEM SINAL, PRONTO, AGUARDANDO	Estados explícitos reduzem ambiguidades e permitem compreender instantaneamente a condição do sistema.
Análise	Gráficos temporais, indicadores de desempenho, matriz de confusão, histórico de eventos	A visualização gráfica facilita a identificação de padrões em comparação a tabelas extensas.
Controle	Detecção de capacete, colete, máscara e óculos	Controles próximos às áreas de monitoramento reduzem o deslocamento visual do usuário.

Severidade dos alertas. A interface segue uma escala progressiva de criticidade, de informativa a crítica:

Severidade	Situações (exemplos)	Objetivo
Baixa	Sistema inicializando; carregamento e atualização de dados	Informar sem interromper o fluxo de trabalho.
Média	Perda temporária de comunicação; oscilações de desempenho; queda momentânea da precisão	Permitir ação preventiva antes que o problema se torne crítico.
Alta	Câmera offline; falha contínua de comunicação; ausência de detecção em áreas monitoradas	Garantir rápida identificação e correção.
Crítica	Falha geral de monitoramento; ausência de EPIs em área de risco; falha simultânea em múltiplas câmeras	Priorizar imediatamente a atenção do operador.

Essa escala (Baixa → Média → Alta → Crítica) é a base da hierarquia visual de alertas (Normal → Atenção → Alto → Crítico) que pode ser construída na interface, mantendo coerência entre a lógica de severidade e a evolução do produto.

Linguagem. Termos curtos, mensagens diretas, baixa complexidade textual, padronização dos estados e redução de ambiguidades. Exemplos: “A câmera não está enviando dados no momento” → **SEM SINAL**; “O sistema está aguardando uma conexão” → **AGUARDANDO**; “O equipamento encontra-se em funcionamento” → **PRONTO**.

MENTORIA 4 · TELAS DE AUDITORIA E INVENTÁRIO DE INCIDENTES
Proposta de novas telas, alinhadas à escala de severidade e à linguagem acima.

- **Inventário de incidentes:** lista paginada com data/hora, câmera ou zona, tipo de risco (EPI, ergonomia, zona crítica, queda), severidade, EPI ausente ou score REBA, confiança e status; com filtros por período, zona, tipo, severidade e status, além de busca;
- **Detalhe do incidente:** frame de evidência com as bounding boxes, metadados da detecção e linha do tempo do tratamento;
- **Fluxo de tratamento (auditoria):** status (por exemplo, Aberto → Em análise → Resolvido), com registro de quem alterou, quando e uma justificativa, de forma que o histórico não possa ser apagado pelo usuário comum;
- **Indicadores para RH/CIPA:** incidentes por tipo, zona e turno, reincidência e tempo médio de tratamento, reaproveitando os componentes de gráfico do dashboard;
- **Exportação:** relatório de auditoria em PDF/CSV, ampliando o botão de geração de relatório já existente.

3.1.5 Integração com backend

O front-end consome a API Flask para receber, em tempo real, as classificações produzidas pelo modelo de ML e alimentar o painel de detecção ao vivo, os gráficos e a central de logs, tratando os diferentes estados de resposta (detecção confirmada, ausência de EPI, erro de conexão). O resultado por frame é transmitido via WebSocket; os incidentes confirmados são consultados pela API REST. As novas telas de auditoria consomem os endpoints de incidentes descritos na seção 4.1.3.

> **MENTORIA 4 · NOTIFICAÇÃO VISUAL E SONORA**
> O frontend continua responsável pelo alerta visual e sonoro na tela do operador; a sinalização física com ESP32 (seção 4.1.5) usa a mesma escala de severidade, de modo que tela e dispositivo comuniquem o mesmo nível de alerta.

3.1.6 Ferramentas

Categoria	Tecnologia utilizada
Linguagem	JavaScript
Framework	React 19
Build	Vite
Estilização	TailwindCSS v4
Gerenciamento de estado	Zustand
Dashboards e gráficos	Recharts
Comunicação com a API	Axios
Prototipação e identidade visual	Figma

A combinação de React com Zustand centraliza o estado das câmeras, detecções e indicadores em um único ponto da aplicação, sustentando a decisão de UX de reunir monitoramento, controle e análise em uma única interface. A Axios padroniza a comunicação com a API que entrega os estados operacionais e os dados dos gráficos, e o Tailwind CSS viabiliza a aplicação consistente das cores de status e da hierarquia visual.

3.1.7 Pipeline de implementação

O desenvolvimento seguiu a sequência: definição da identidade visual no Figma; tela web/index; dashboard inicial e aba de logs; atualização incremental de gráficos conforme a evolução do modelo; e tela de câmera de zona de perigo. Nesta etapa acrescentam-se as telas de inventário e auditoria de incidentes.

3.2 Resultados

3.2.1 Critérios de avaliação

A interface foi avaliada de forma preliminar quanto à clareza dos alertas visuais e sonoros de não conformidade, à legibilidade das bounding boxes sobrepostas ao vídeo e à organização das informações no dashboard para consulta rápida por gestores de SST. As telas de auditoria devem ser avaliadas quanto à rapidez para localizar um incidente e à clareza do fluxo de tratamento.

3.2.2 Funcionalidades entregues

- Tela de câmera ao vivo com detecção e rotulagem em tempo real;
- Painel de seleção de EPIs a monitorar;
- Painel de atenção com os percentuais de confiança;
- Dashboard completo com os cinco cards de KPIs e central de logs com botão de geração de relatório;
- Estados operacionais padronizados e escala de severidade documentada.

3.2.3 Análise de problemas

Como pontos de atenção, destacam-se a necessidade de testes de usabilidade mais formais com usuários representativos (equipes de RH/CIPA) e o refinamento do comportamento do painel de alertas em cenários com múltiplas não conformidades simultâneas. A hierarquia visual avançada de alertas, o score de risco contextualizado e a visualização da estimação de pose corporal permanecem como evoluções futuras.

3.3 Evidências

3.3.1 Prints e protótipos de telas

As telas implementadas refletem o protótipo definido no Figma, com a tela de câmera ao vivo e o painel lateral de detecção de EPIs.



Figura 4 - Tela de câmera ao vivo com painel de detecção de EPIs e painel de atenção.

3.3.2 Exemplos práticos

O dashboard de KPIs (Figura 1) ilustra o fluxo completo de uso: consulta de logs, análise de gráficos de detecção e geração de relatório a partir da interface.

3.3.3 Logs e reprodutibilidade

O código do front-end é versionado no GitHub do grupo, com o planejamento de sprints documentado no Miro. A configuração do ambiente segue as dependências do React e das bibliotecas de visualização dos gráficos.

4. Backend

4.1 Desenvolvimento

4.1.1 Revisão teórica

A frente de Backend reaproveita o padrão de arquitetura validado no Challenge Aché 2025 — API em Flask com persistência em SQLite —, adaptado para receber as inferências do modelo de visão computacional e expor os dados necessários ao dashboard e à central de logs.

4.1.2 Requisitos e escopo dos serviços

O escopo do backend contempla: setup do Flask; criação da API e do banco de dados; sistema de cadastro/login; integração dos logs com os gráficos do dashboard; controle das configurações do modelo de ML (parâmetros de inferência e classes monitoradas); e, na Mentoria 4, o inventário de incidentes com trilha de auditoria e a integração com o ESP32.

4.1.3 Arquitetura

A arquitetura é composta por uma API Flask que expõe endpoints de inferência e consulta, com persistência em SQLite. Um módulo de Sistema de Logs armazena as detecções no banco, guarda os frames em arquivo local, disponibiliza endpoints de consulta de logs e registra eventos de zona de perigo e riscos ergonômicos. O orquestrador transmite o resultado em tempo real ao frontend via WebSocket e registra incidentes confirmados via API REST.

> **MENTORIA 4 · INVENTÁRIO DE INCIDENTES E AUDITORIA**
> Para sustentar as telas da seção 3.1.4, o backend passa a tratar o incidente como entidade própria.

- **Tabela de incidentes:** identificador, data/hora, câmera, zona, tipo de risco, severidade, EPI ausente ou score REBA, confiança, caminho do frame de evidência e status;
- **Tabela de histórico de auditoria:** cada mudança de status ou observação registrada com usuário, data/hora e justificativa, apenas com inserção (sem edição ou exclusão);
- **Endpoints (nomes ilustrativos):** listagem com filtros e paginação, detalhe do incidente, atualização de status e exportação em PDF/CSV;
- **Controle de acesso:** aproveitando o sistema de cadastro/login, com acesso às evidências restrito a perfis autorizados, conforme as diretrizes de LGPD.

4.1.4 Integração com o modelo de ML

O backend recebe as saídas dos modelos (classes detectadas, bounding boxes, scores de confiança e resultados de pose e zona) por meio do módulo de inferência, processa essas informações, grava os eventos na base de dados e disponibiliza os dados agregados para os endpoints consumidos pelo dashboard e pela central de logs. A confirmação de incidentes usa o debounce definido na seção 2.1.5, evitando registros duplicados por leituras ruidosas.

4.1.5 Integração com hardware (ESP32)

MENTORIA 4 · NOTIFICAÇÃO VISUAL E SONORA COM COMPONENTES FÍSICOS
Além do alerta na tela, o sistema passa a acionar um dispositivo físico baseado em ESP32, instalado próximo ao posto monitorado, com sinalização luminosa e sonora proporcional à severidade do incidente.

Arquitetura proposta. O ESP32 conecta-se à mesma rede local do servidor Flask e recebe comandos quando um incidente é confirmado (após o debounce), por HTTP ou por protocolo de mensagens leve (como MQTT ou WebSocket). O firmware interpreta a severidade recebida e aciona os atuadores: LED (ou fita/LED RGB) para a sinalização visual e buzzer para a sinalização sonora. Quando o incidente é encerrado, o backend envia o comando de retorno ao estado normal.

Mapeamento de severidade (proposta a validar em testes):

Evento	Severidade	Sinal visual (LED)	Sinal sonoro (buzzer)
Operação normal / sistema pronto	Baixa	Verde ou azul fixo	Sem som
EPI ausente fora de zona de risco; ergonomia REBA médio	Média	Amarelo fixo	Bipe curto único
Ergonomia REBA alto; invasão de zona; câmera offline	Alta	Laranja piscando	Bipes intermitentes
Queda; ausência de EPI em área de risco	Crítica	Vermelho piscando rápido	Tom contínuo/sirene

- **Estados do dispositivo:** o ESP32 envia sinal periódico de vida (heartbeat); se ele parar de responder, o dashboard exibe SEM SINAL para o dispositivo, reutilizando o vocabulário de estados da seção 3.1.4;
- **Prevenção de fadiga de alarme:** uso dos cooldowns já definidos por tipo de risco e possibilidade de silenciar o som pelo operador, mantendo o sinal luminoso;
- **Limite de escopo:** o dispositivo apenas sinaliza; não comanda esteiras, catracas ou PLCs, sem qualquer integração com o processo produtivo;
- **Firmware:** linguagem e biblioteca de comunicação a definir pelo grupo (por exemplo, C++ com Arduino ou MicroPython).

4.1.6 Ferramentas

- **Backend:** Node.js · Express · SQLite · JWT (autenticação) · ExcelJS · PDFKit · Puppeteer (relatórios) · Ollama (chatbot de IA);
- **Tempo real:** WebSocket (porta 8765), entre o orquestrador e o frontend;
- **Apoio ao desenvolvimento:** Git / GitHub (controle de versão e branches por feature) · VS Code · Miro · Trello · documentação de requisitos e UML versionada no repositório · scripts de smoke test manuais;
- **Sinalização física:** ESP32 com LED e buzzer (Mentoria 4).

4.1.7 Pipeline de implementação

O desenvolvimento seguiu duas frentes paralelas: (1) API Central — setup do Flask, criação da API e do banco de dados, cadastro/login, integração dos dados de logs com os gráficos e controle das configurações do modelo de ML; e (2) Sistema de Logs — armazenamento no banco, frames em arquivo local, endpoints de logs e registros específicos de zona de perigo e riscos ergonômicos. Na Mentoria 4 acrescentam-se: modelagem de incidentes e auditoria, integração com o ESP32 e instrumentação de latência.

4.2 Resultados

4.2.1 Métricas de avaliação

As métricas de desempenho dos serviços — tempo de resposta dos endpoints, disponibilidade e taxa de erro — serão medidas de forma sistemática com a API integrada de ponta a ponta ao pipeline de inferência e ao dashboard. Para a Mentoria 4, acrescentam-se a latência por etapa do streaming (seção 2.1.5) e o tempo entre a confirmação do incidente e o acionamento do ESP32.

4.2.2 Desempenho dos serviços

Até aqui, o foco esteve na criação da estrutura da API e do banco de dados. Testes de carga e de performance, bem como a medição da latência ponta a ponta, ainda devem ser executados e reportados.

4.2.3 Análise de erros

Não há, até o momento, falhas críticas documentadas nos serviços de backend. A análise de erros será aprofundada com o aumento do volume de requisições e com a entrada do dispositivo físico (falhas de rede, perda de conexão do ESP32).

4.3 Evidências

4.3.1 Visualizações de suporte

O fluxograma abaixo representa o fluxo completo do processo, com destaque para a camada de Sistema de Log, responsável por registrar os dados de detecção, salvar os frames e armazenar as informações no banco de dados.

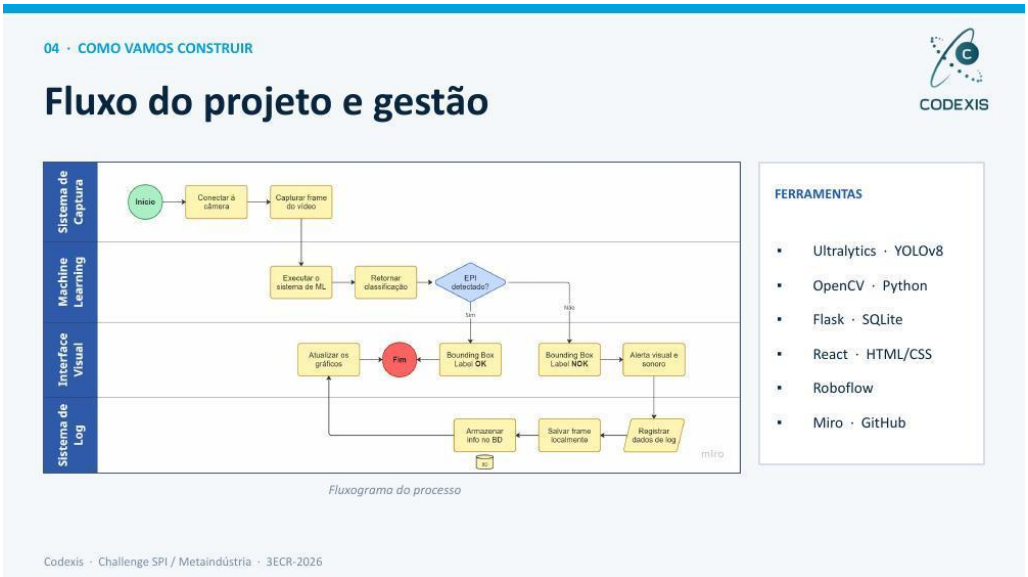


Figura 5 - Fluxograma do processo: captura, inferência, interface visual e sistema de log.

4.3.2 Exemplos práticos

A central de logs exibida no dashboard (Figura 1) é alimentada diretamente pelos endpoints da API, evidenciando o funcionamento integrado entre backend, modelo de ML e front-end — cada linha de log corresponde a um evento processado e persistido pelo backend.

4.3.3 Logs e reprodutibilidade

O código do backend é versionado no GitHub do grupo (branch de referência da Fase 2: ml/ergonomia-zona-risco-labels). A configuração do ambiente, variáveis e parâmetros do banco de dados serão documentados em detalhe no repositório à medida que o setup evoluir, incluindo a configuração do firmware do ESP32.

5. Conclusão

5.1 Considerações finais

A solução proposta pelo grupo Codexis endereça uma dor real e mensurável do setor industrial brasileiro, substituindo a fiscalização manual e reativa por um monitoramento contínuo e automatizado do uso de EPIs, da postura e da presença em zonas críticas — prevendo o risco, e não apenas detectando a infração. Como perspectivas de evolução, destacam-se: a adoção de edge computing, com inferência embarcada em câmeras IP industriais; a melhoria contínua do modelo com imagens reais de chão de fábrica; a expansão da análise comportamental (riscos ergonômicos, aglomerações e invasões de zonas restritas); o score de risco contextualizado; e a fusão multicâmera.

Do ponto de vista de impacto prático, a solução tem potencial para reduzir a ocorrência de acidentes de trabalho relacionados a EPIs, diminuir os custos associados à SST e fornecer subsídios objetivos e rastreáveis para a tomada de decisão de equipes de RH e CIPA. Com a Mentoria 4, esse ciclo se completa: o sistema detecta, alerta no local por sinalização física, registra o incidente e permite auditar seu tratamento.